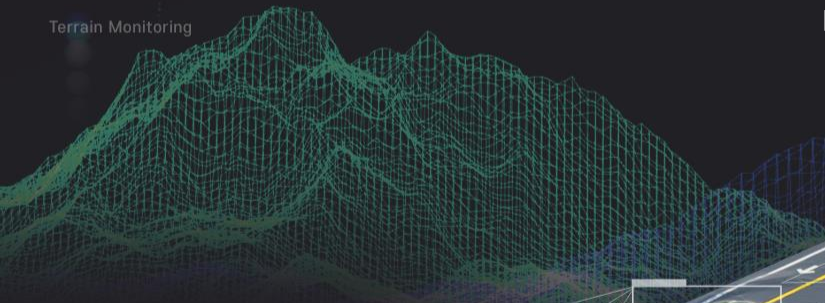


Image Recognition



Terrain Monitoring



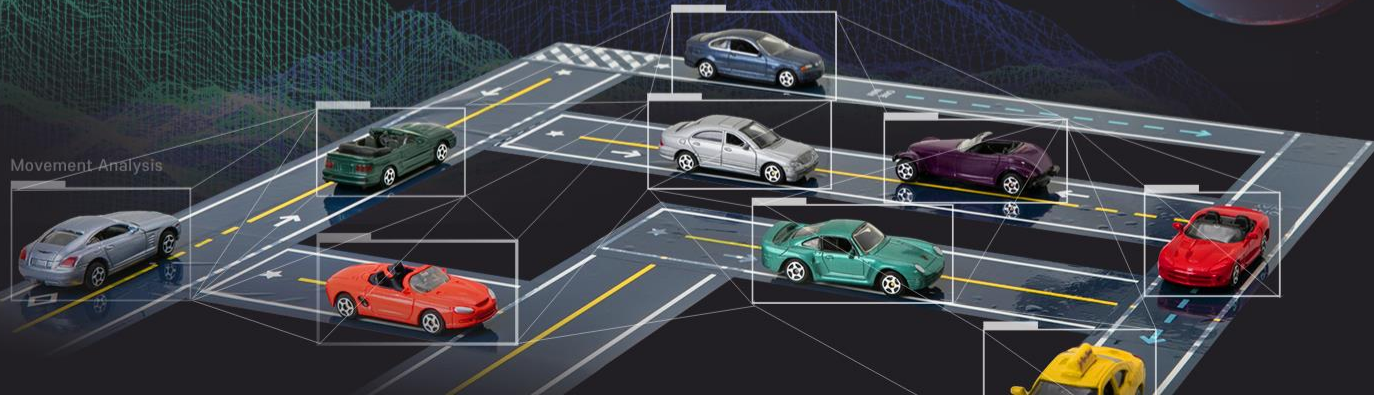
강원혁신플랫폼

컴퓨터 비전

◆
텐서플로의 이해

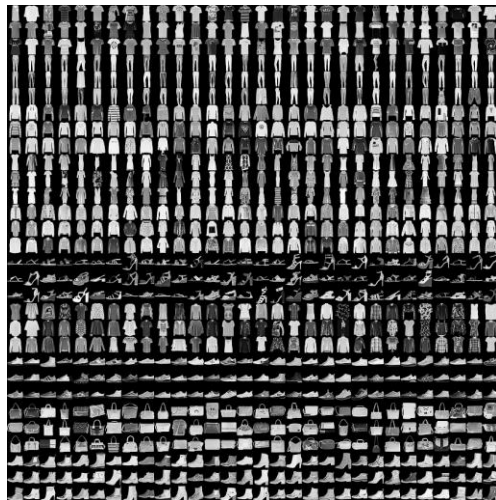
Autonomous Driving

Movement Analysis



패션 MNIST 데이터셋을 이용한 2층 완전연결층 인공지능망 구조를 만들어 보자.

패션 MNIST는 운동화나 셔츠같은 옷과 신발의 이미지와 이미지에 대한 레이블을 제공함



- 훈련용 데이터는 6만 장이며 28×28 픽셀 크기로 저장되어 있음
- 테스트 데이터는 1만 장의 이미지가 준비되어 있음

Fashion MNIST 이미지 데이터셋

패션 MNIST 데이터셋을 이용한 2층 완전연결층 인공지능망 구조를 만들어 보자.

패션 MNIST 데이터셋의 이미지는 28×28 크기의 넘파이 배열이고 픽셀 값은 0~255 사이임
레이블(Label)은 0~9까지의 정수 배열(이 값은 이미지에 있는 옷의 클래스(Class)를 나타냄)

레이블	클래스
0	T-shirt/top(티셔츠)
1	Trouser(바지)
2	Pullover(풀오버 상의)
3	Dress(여성용 드레스)
4	Coat(외투)
5	Sandal(샌들)
6	Shirt(셔츠)
7	Sneaker(스니커 운동화)
8	Bag(가방)
9	Ankle boot(앵클 부츠)





패션 MNIST 데이터는 keras의 데이터셋에 있는데
이를 읽어와서 훈련용 데이터와 테스트 데이터로 구분한다.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

print(train_images.shape) # 학습 이미지의 형상 : (60000, 28, 28)
print(train_labels)      # 학습 이미지의 레이블 : [9 0 0 ... 3 0 5]
print(test_images.shape) # 테스트 이미지의 형상 : (10000, 28, 28)
```



훈련용 데이터에서 첫 번째 이미지의 픽셀 값을 출력한다.

```
print("형상:", train_images[0].shape) # (28, 28)
print(train_images[0])
```

```
[[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  13  73  0
   0  1  4  0  0  0  0  0  1  1  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  3  0  36 136 127  62
   54  0  0  0  1  3  4  0  0  3]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  6  0 102 204 176 134
   144 123  23  0  0  0  0 12 10  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 155 236 207 178
   107 156 161 109  64  23  77 130  72 15]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  1  0  69 207 223 218 216
   216 163 127 121 122 146 141  88 172  66]
 [ 0  0  0  0  0  0  0  0  0  0  1  1  1  0 200 232 232 233 229
   223 223 215 213 164 127 123 196 229  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0 183 225 216 223 228
   235 227 224 222 224 221 223 245 173  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0 193 228 218 213 198
   180 212 210 211 213 223 220 243 202  0]
 [ 0  0  0  0  0  0  0  0  0  0  1  3  0 12 219 220 212 218 192
```

출력 결과 0~255 사이의
픽셀 값인 것을 볼 수 있음

훈련용 데이터에서 첫 번째 이미지의 레이블을 출력한다.

```
print(train_labels[0]) # 9
```

출력 결과 첫 번째 이미지의 레이블은 9이고, 레이블은 0~9까지의 정수 배열인 것을 알 수 있음

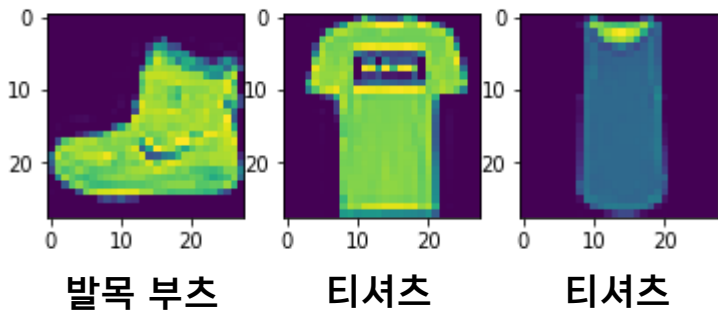
레이블	클래스
0	T-shirt/top(티셔츠)
1	Trouser(바지)
2	Pullover(풀오버 상의)
3	Dress(여성용 드레스)
4	Coat(외투)
5	Sandal(샌들)
6	Shirt(셔츠)
7	Sneaker(스니커 운동화)
8	Bag(가방)
9	Ankle boot(앵클 부츠)

훈련용 데이터에서 첫 번째 ~ 세 번째 이미지를 출력한다.

```
fig = plt.figure()
ax1 = fig.add_subplot(1, 3, 1)
ax2 = fig.add_subplot(1, 3, 2)
ax3 = fig.add_subplot(1, 3, 3)

ax1.imshow(train_images[0]) # 첫 번째 훈련용 이미지
ax2.imshow(train_images[1]) # 두 번째 훈련용 이미지
ax3.imshow(train_images[2]) # 세 번째 훈련용 이미지
plt.show()
print(train_labels[0:3]) # 레이블 = [9 0 0]
```

출력 결과 발목 부츠, 티셔츠, 티셔츠 인 것을 볼 수 있음



레이블	클래스	레이블	클래스
0	T-shirt/top(티셔츠)	5	Sandal(샌들)
1	Trouser(바지)	6	Shirt(셔츠)
2	Pullover(풀오버 상의)	7	Sneaker(스니커 운동화)
3	Dress(여성용 드레스)	8	Bag(가방)
4	Coat(외투)	9	Ankle boot(앵클 부츠)



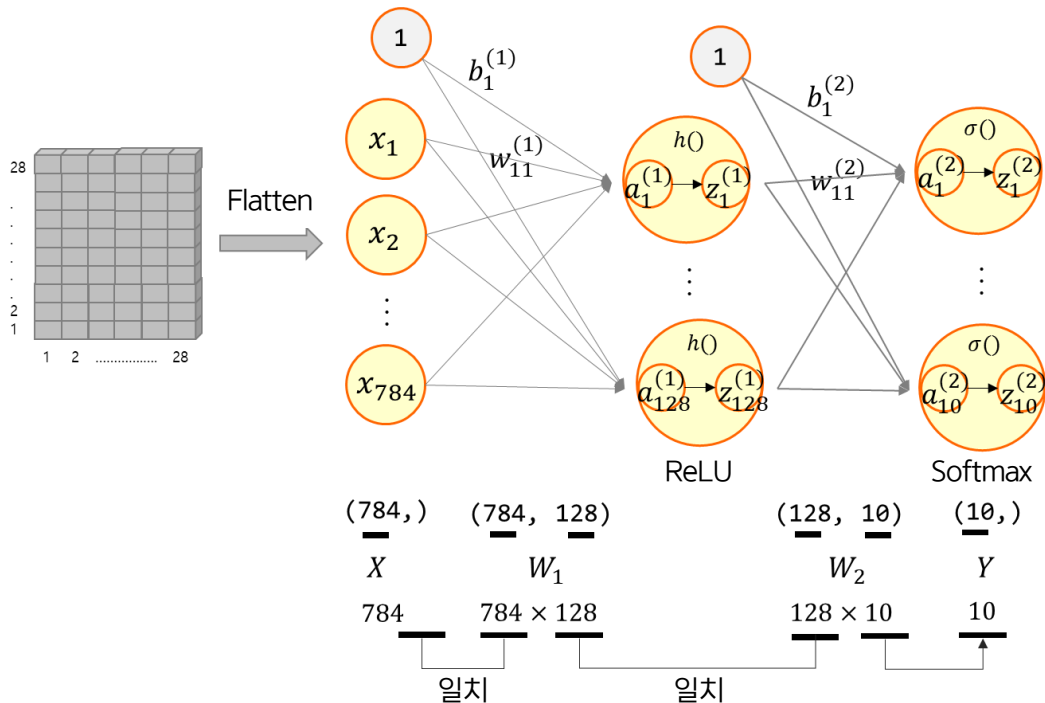
2층 인공신경망 네트워크 구조를 구축한다.

- 텐서플로 케라스의 Dense 클래스는 학습을 위한 연결을 밀집된(Dense) 구조 혹은 완전연결층(Fully-connected)으로 함
- Dense 클래스로 2개의 완전연결층을 가진 네트워크 모델을 생성함
 - 1 Flatten 계층 : 2차 입력 (28, 28)을 1차원 784(28×28)으로 변경
 - 2 Dense 네트워크 : 출력 128, 활성화함수 = ReLU
 - 3 Dense 네트워크 : 출력 10, 활성화함수 = SoftMax

2층 신경망 계층구조

```
model = keras.Sequential([  
    keras.layers.Flatten(input_shape=(28, 28)),  
    keras.layers.Dense(128, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```


2층 인공신경망 네트워크 구조를 구축한다.



인공신경망 각 층의 배열 형상



2층 인공신경망 네트워크 구조를 구축한다.

```
# 2층 신경망 계층구조
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

1 ▶ **Flatten** 네트워크에는 **입력**을 그대로 **한 줄로 만드는 것**이기 때문에 필요한 **매개변수**는 **입력의 크기**임

☞ 앞의 예에서는 **2차원**(28×28) **입력**을 **1차원**(784)으로 **변경**함



2층 인공신경망 네트워크 구조를 구축한다.

```
# 2층 신경망 계층구조
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

2 Dense 네트워크는 학습을 위한 연결을 밀집된(Dense) 구조 혹은 완전연결층(Fully connected)으로 한다는 의미임

- ➡ Dense 네트워크의 입력은 앞 층에서 주어지기 때문에 몇 개의 출력으로 연결할지를 정하는 매개변수가 있음
- ➡ 앞의 예에서는 128 뉴런(은닉층), 10 뉴런(출력층)의 출력으로 연결함
- ➡ 출력 값을 결정하는 활성화 함수를 정해줌
- ➡ 앞의 예에서는 활성화 함수로 ReLU(은닉층), SoftMax(출력층)를 사용함

텐서플로를 이용해 2층 인공신경망의 계층 구조로 생성한 모델을 학습시켜 보자.

다음은 `compile()` 함수로 모델 학습 과정을 설정하는 파이썬 코드이다.

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

- ⊕ 신경망 학습은 기본적으로 추측을 한 뒤에 정답과 비교하여 오차가 얼마인지 확인한 뒤에 이 오차를 줄이는 방법으로 연결의 강도를 조절하는 것임
- ⊕ 모델을 점점 더 좋은 상태로 만드는 것을 최적화(Optimization)라고 부름
(위 예제에서는 아담(Adam)을 사용)
- ⊕ 현재 모델의 정답과 실제 정답의 차이가 오차이고, 이 오차를 측정하는 것이 손실함수
(위 예제에서는 `sparse_categorical_crossentropy` 함수를 사용)
- ⊕ 위 코드에서는 모델 평가 지표로 정확도(Accuracy)를 사용

텐서플로를 이용해 2층 인공신경망의 계층 구조로 생성한 모델을 학습시켜 보자.

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

keras.Model.compile() 함수의 중요한 세 개의 파라미터

Loss	최적화 과정에서 최소화될 손실함수를 설정하는 것(MSE 등)
Optimizer	훈련 과정을 설정하는 것(Adam, SGD 등)
Metrics	훈련을 모니터링하기 위해 사용(Accuracy 등)



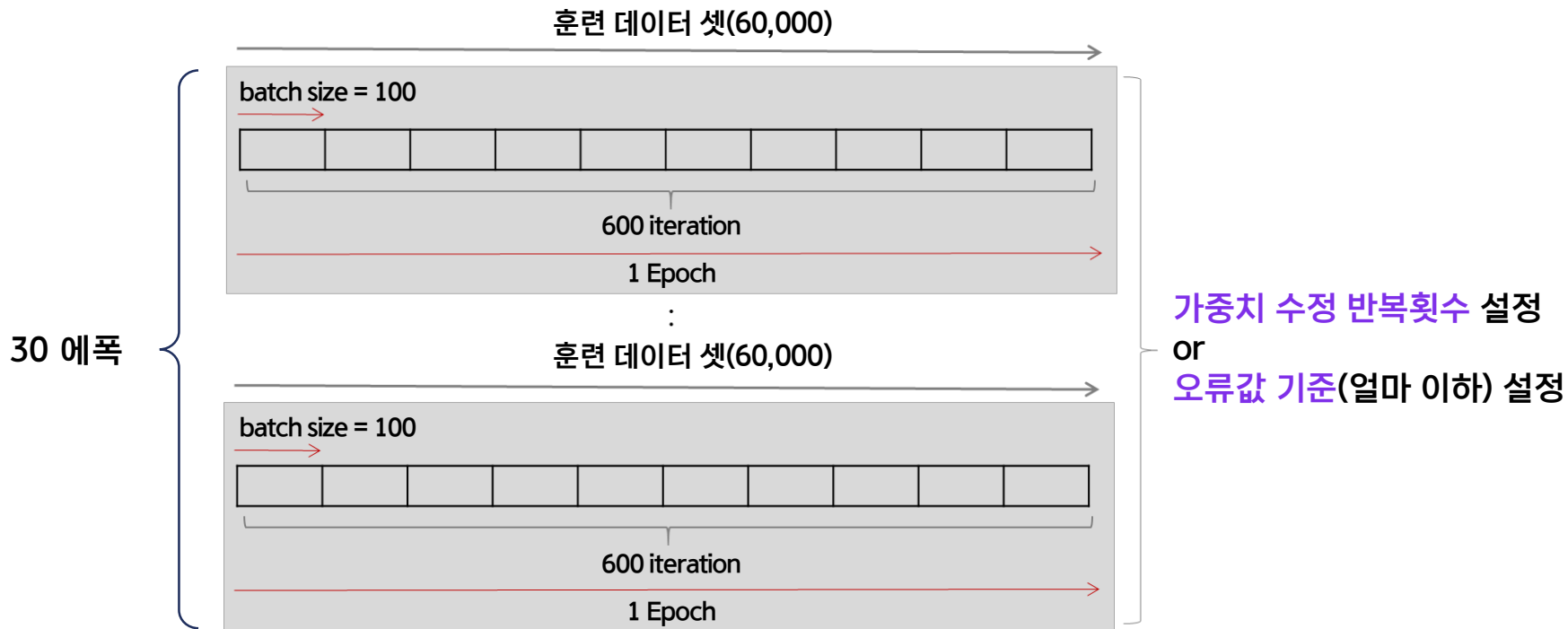
텐서플로를 이용해 2층 인공신경망의 계층 구조로 생성한 모델을 학습시켜 보자.

다음은 `fit()` 함수로 모델을 학습시키는 파이썬 코드이다.

```
model.fit(train_images, train_labels, epochs=30)
```

- 모델이 완성되면 훈련 데이터와 정답 데이터를 주고 학습을 실시함
- 학습을 시작하게 하는 함수는 모델의 `fit()` 메소드인데, 훈련용 입력과 여기에 대응하는 정답 레이블 데이터 셋을 차례로 설정하면 됨
- 훈련 데이터 모음을 가지고 한 번 훈련을 실시하는 것을 에폭(Epoch)이라고 부름
- 위 코드에서는 30개의 에폭을 지정하였으므로 훈련을 30차례 실시하는 것임

텐서플로를 이용해 2층 인공신경망의 계층 구조로 생성한 모델을 학습시켜 보자.



텐서플로를 이용해 2층 인공신경망의 계층 구조로 생성한 모델을 학습시켜 보자.

다음은 `fit()` 함수로 모델을 학습시킨 결과이다.

```
hist = model.fit(train_images, train_labels, epochs=30)
```

```
Epoch 25/30  
1875/1875 [=====] - 6s 3ms/step - loss: 0.4316 - accuracy: 0.8526  
Epoch 26/30  
1875/1875 [=====] - 6s 3ms/step - loss: 0.4204 - accuracy: 0.8543  
Epoch 27/30  
1875/1875 [=====] - 6s 3ms/step - loss: 0.4218 - accuracy: 0.8548  
Epoch 28/30  
1875/1875 [=====] - 5s 3ms/step - loss: 0.4305 - accuracy: 0.8535  
Epoch 29/30  
1875/1875 [=====] - 6s 3ms/step - loss: 0.4191 - accuracy: 0.8554  
Epoch 30/30  
1875/1875 [=====] - 6s 3ms/step - loss: 0.4271 - accuracy: 0.8529
```

위 결과는 모델로 훈련 데이터를 이용해 30에폭 학습을 수행한 결과임

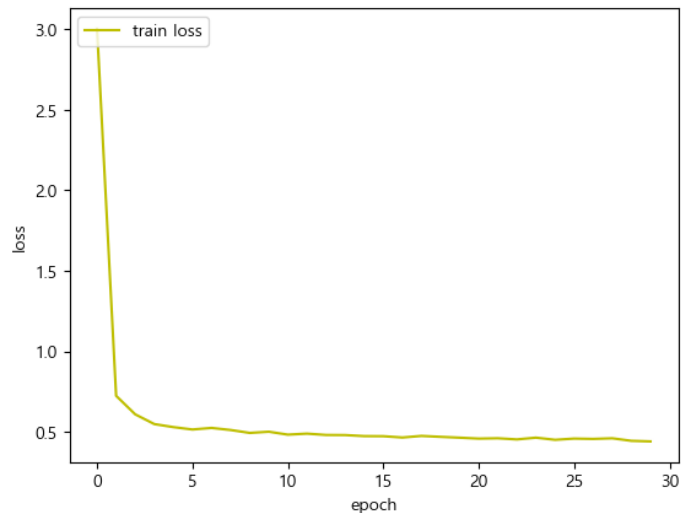
텐서플로를 이용해 2층 인공신경망의 계층 구조로 생성한 모델을 학습시켜 보자.

다음은 모델로 훈련 데이터를 이용해 학습한 결과를 시각화하는 파이썬 코드이다.

```
import matplotlib.pyplot as plt

fig, loss_ax = plt.subplots()
loss_ax.plot(hist.history['loss'], 'y', label='train loss')
loss_ax.set_xlabel('epoch')
loss_ax.set_ylabel('loss')
loss_ax.legend(loc='upper left')
plt.show()
```

위 결과를 살펴보면
모델이 훈련 데이터를 잘 학습하고 있음을 볼 수 있음



테스트 데이터로 모델 평가를 수행해보자.

다음은 `evaluate()` 함수로 학습에서 얻은 모델을 테스트 데이터로 평가를 수행하는 파이썬 코드이다.

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)

print('\n테스트 정확도:', test_acc) # 테스트 정확도: 0.8137000203132629
```

실행 결과 정확도는 81.37% 인 것을 알 수 있음



학습된 2층 신경망 모델을 저장한다.

다음은 `save()` 함수로 학습된 모델을 저장하는 파이썬 코드이다.

```
import os

model.save(os.getcwd()+'/model/fashion_dnn2_model.h5')
print("Saved model to disk !!!")
```

위 코드 수행결과 학습된 모델이 작업경로 아래의
'/model/fashionon_dnn2_model.h5' 파일로 저장된 것을 알 수 있음



저장된 2층 신경망 모델을 읽어온다.

다음은 `load_model()` 함수로 저장된 2층 신경망 모델을 읽어오는 파이썬 코드이다.

```
import os
from tensorflow.keras.models import load_model

model = load_model(os.getcwd()+'/model/fashion_dnn2_model.h5')
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 10)	1290

=====
Total params: 101,770
Trainable params: 101,770
Non-trainable params: 0
=====

테스트 데이터에서 무작위로 하나의 이미지를 선택해 출력해보자.

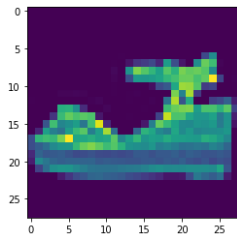
다음은 `np.random.randint()` 함수로 테스트 데이터에서 무작위로 하나의 이미지를 선택해서 출력하는 파이썬 코드이다.

```
test_images.shape      # (10000, 28, 28)

import numpy as np

randIdx = np.random.randint(0, 10000)
plt.imshow(test_images[randIdx])

print(test_labels[randIdx])    # 5 : Sandal (샌달)
```



샌들

레이블	클래스	레이블	클래스
0	T-shirt/top(티셔츠)	5	Sandal(샌들)
1	Trouser(바지)	6	Shirt(셔츠)
2	Pullover(풀오버 상의)	7	Sneaker(스니커 운동화)
3	Dress(여성용 드레스)	8	Bag(가방)
4	Coat(외투)	9	Ankle boot(앵클 부츠)

테스트 데이터에서 무작위로 하나의 이미지를 선택해 모델로 예측해보자.

다음은 테스트 데이터에서 무작위로 하나의 이미지를 선택해서 모델로 예측하는 파이썬 코드이다.

```
print(test_images[randIdx].shape)      # 하나의 이미지 형상 : (28, 28)
yhat = model.predict(test_images[randIdx])
```

실행 결과 아래와 같은 오류가 발생하는 것을 볼 수 있음

```
ValueError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_17724\364016168.py in <module>
      3 print(test_images[randIdx].shape)      # 하나의 이미지 형상 : (28, 28)
      4
--> 5 yhat = model.predict(test_images[randIdx])
      6
      7 # 실행 결과 오류가 발생한다.

~\anaconda3\lib\site-packages\keras\utils\traceback_utils.py in error_handler(*args, **kwargs)
     68     # To get the full stack trace, call:
     69     # `tf.debugging.disable_traceback_filtering()`
--> 70     raise e.with_traceback(filtered_tb) from None
     71 finally:
     72     del filtered_tb

~\anaconda3\lib\site-packages\keras\engine\training.py in tf_predict_function(iterator)
     13     try:
     14         do_return = True
--> 15         retVal = ag__.converted_call(ag__.ld(step_function), (ag__.ld(self), ag__.ld(iterator)), None, fscope)
     16     except:
     17         do_return = False

ValueError: in user code:
```

테스트 데이터에서 무작위로 하나의 이미지를 선택해 모델로 예측해보자.

다음은 테스트 데이터에서 무작위로 하나의 이미지를 선택해서 모델로 예측하는 파이썬 코드이다.

```
print(test_images[randIdx].shape)      # 하나의 이미지 형상 : (28, 28)
yhat = model.predict(test_images[randIdx])
```

- ❶ 위 코드에서 하나의 **이미지 형상은 (28, 28)**이지만, 모델의 **신경망 구조에서 3차원 배열** 입력을 **처리하는 모델**을 만들었음
- ❷ 그러므로 모델에 **입력하는 이미지의 형상**을 2차원(28,28)에서 **3차원(1,28,28)**으로 **변경하여야 함**

Numpy 배열(array)의 차원을 늘여주는 방법에 대해 살펴보자.

numpy.newaxis 함수

- numpy 배열의 차원을 확장하는 함수
- 형상(Shape)은 변환 전 차원의 합과 변환 후 차원의 합이 같아야 함

```
arr = np.arange(4)
print(arr.shape)    # (4,)

# make it as row vector by inserting an axis along first dimension
row_vec = arr[np.newaxis, :]
print(row_vec.shape) # (1, 4)

# # make it as column vector by inserting an axis along second dimension
col_vec = arr[:, np.newaxis]
print(col_vec.shape) # (4, 1)
```

위 코드에서 변환 전과 변환 후의 차원의 합이 동일한 것을 알 수 있음



학습된 2층 신경망 모델로 테스트 데이터에 있는 새 이미지에 적용해 예측을 수행해보자.

다음은 `predict()` 함수로 테스트 데이터에서 무작위로 선택된 하나의 이미지를 예측하는 파이썬 코드이다. 임의로 선택한 이미지의 형상을 (1,28,28)으로 변경하여 예측을 수행한다.

```
yhat = model.predict(test_images[randIdx][np.newaxis, :, :]) # 이미지 형상 : 3차원 늘려 모델에 입력
print(yhat)

# 클래스를 찾아 출력하게 만들기
yhat = np.argmax(model.predict(test_images[randIdx][np.newaxis, :, :]))
print(yhat)          # 5

# 클래스 이름을 리스트로 만들기
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

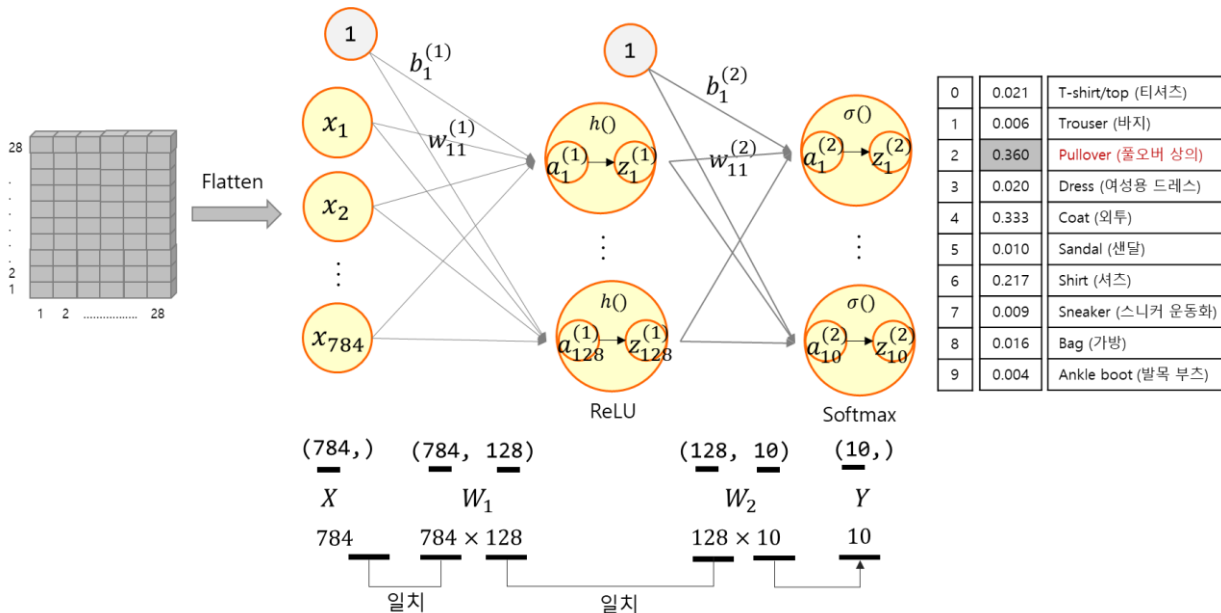
yhat = np.argmax(model.predict(test_images[randIdx][np.newaxis, :, :]))
print("예측:", class_names[yhat])      # 예측: Sandal
print("실제:", class_names[test_labels[randIdx]]) # 실제: Sandal
```

```
1/1 [=====] - 0s 20ms/step
[[1.0381722e-34 3.5867478e-34 2.6246097e-35 0.0000000e+00 0.0000000e+00
 9.9994230e-01 0.0000000e+00 5.6078090e-05 1.5960415e-12 1.6915907e-06]]
1/1 [=====] - 0s 18ms/step
5
1/1 [=====] - 0s 21ms/step
예측: Sandal
실제: Sandal
```

실행결과에서 예측값과 실제값이
일치한 것을 볼 수 있음

학습된 2층 신경망 모델로 테스트 데이터에 있는 새 이미지에 적용해 예측을 수행해보자.

학습된 2층 신경망 모델로 새 이미지에 적용해 예측하는 과정



인공신경망 각 층의 배열 형상



발목 부츠 이미지 파일을 읽어서 화면에 출력해보자.

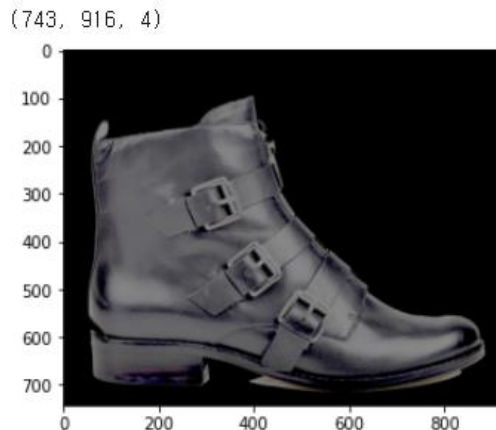
다음은 테스트 데이터에서 무작위로 하나의 이미지를 선택해서 모델로 예측하는 파이썬 코드이다.

```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import os

image1 = mpimg.imread(os.getcwd()+'/images/ankle_boot_reverse.png')
plt.imshow(image1)

print(image1.shape) # (743, 916, 4)
# 형상:(743, 916, 4)(높이,가로,채널) - 불투명도 채널까지 포함된 4-채널 이미지
```

발목 부츠 이미지는 (높이,가로,채널)(743,916,4) 형상의
이미지인 것을 알 수 있음



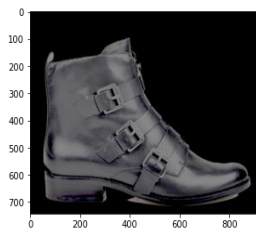


발목 부츠 이미지를 좌우 전환하여 화면에 출력해보자.

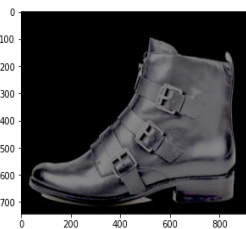
다음은 발목 부츠 이미지를 좌우 전환하는 파이썬 코드이다.

```
# 이미지 상하, 좌우 대칭
# image.transpose(Image.direction*)
# FLIP_LEFT_RIGHT - 좌우 대칭할 경우 입력해줍니다.
# FLIP_TOP_BOTTOM - 상하 대칭일 경우 입력해줍니다.
flp_img = image1.transpose(Image.FLIP_LEFT_RIGHT)

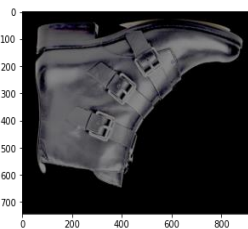
# image.transpose(Image.Rotate_degree*)
# ROTATE_90 - 좌측으로 90도 회전
# ROTATE_180 - 좌측으로 180도 회전
# ROTATE_270 - 좌측으로 270도 회전
flp_img = image1.transpose(Image.ROTATE_90)
```



원본



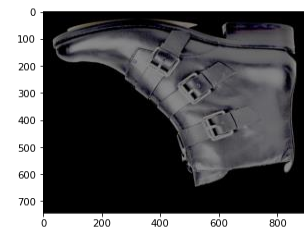
좌우대칭



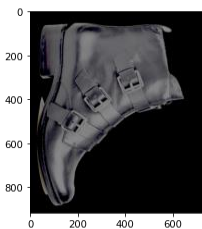
상하대칭



좌측 90도 회전

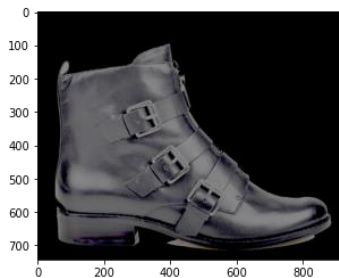


좌측 180도 회전

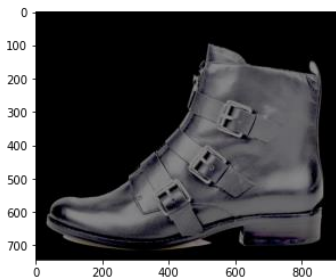


좌측 270도 회전

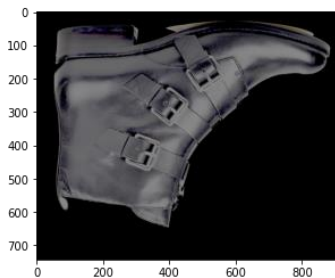
발목 부츠 이미지를 좌우 전환하여 화면에 출력해보자.



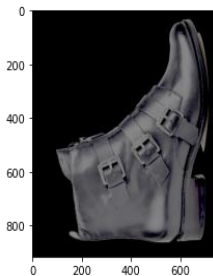
원본



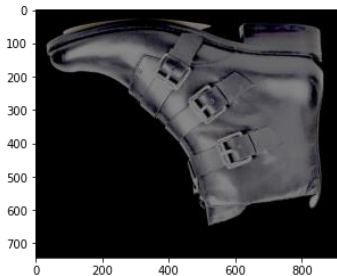
좌우대칭



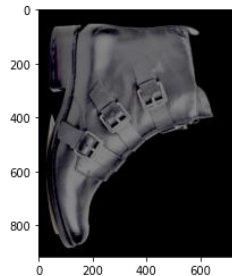
상하대칭



좌측 90도 회전



좌측 180도 회전



좌측 270도 회전

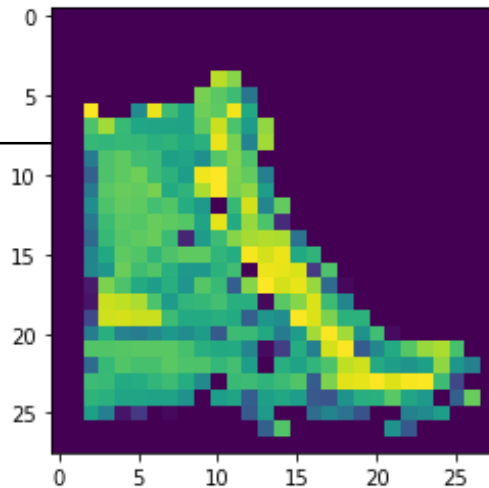
학습된 2층 신경망 모델로 발목 부츠 이미지를 입력하여 예측해보자(원본 이미지)

다음은 opencv를 이용해 발목 부츠 이미지를 회색조 단일 채널로 변환하여 출력하는 파이썬 코드이다.

```
import cv2

img = cv2.imread(os.getcwd()+'/images/ankle_boot_reverse.png', cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (28, 28))
plt.imshow(img)

print(img.shape)    # 형상 : (28, 28)
```



학습된 2층 신경망 모델로 발목 부츠 이미지를 입력하여 예측해보자(원본 이미지)

다음은 발목 부츠 **이미지**의 **현상**(2차원(28,28) → 3차원(1,28,28))을 **변경**하는 파이썬 코드이다.

```
input_data = img[np.newaxis, :, :]  
input_data.shape      # (1, 28, 28)
```

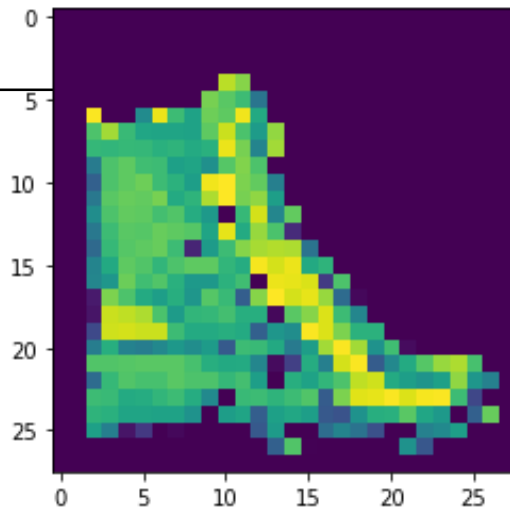
- ⦿ np.newaxis 함수는 numpy 배열(array)의 차원을 늘려줌
- ⦿ 위 코드 실행결과 2차원에서 3차원으로 형상이 변경된 것을 볼 수 있음

학습된 2층 신경망 모델로 발목 부츠 이미지를 입력하여 예측해보자(원본 이미지)

다음은 모델에 발목 부츠 이미지를 적용하여 예측을 수행하는 파이썬 코드이다.

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']  
yhat = np.argmax(model.predict(input_data))  
print(class_names[yhat]) # Bag
```

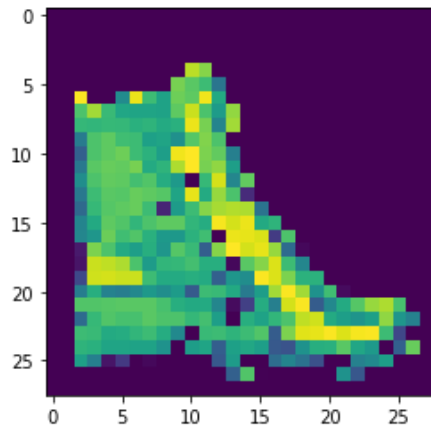
실행 결과, 발목 부츠' 이미지를
가방(Bag)으로 잘못 예측한 것을 알 수 있음



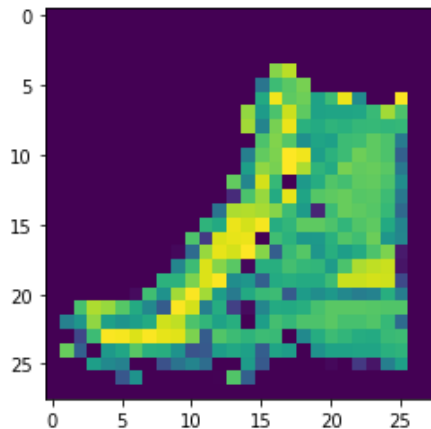
학습된 2층 신경망 모델로 발목 부츠 이미지를 입력하여 예측해보자(원본 이미지)

다음은 발목 부츠 이미지를 좌우 전환하여 화면에 출력하는 코드이다.

```
input_mirror = input_data[:, :, ::-1]  
plt.imshow(input_mirror[0])
```



원본



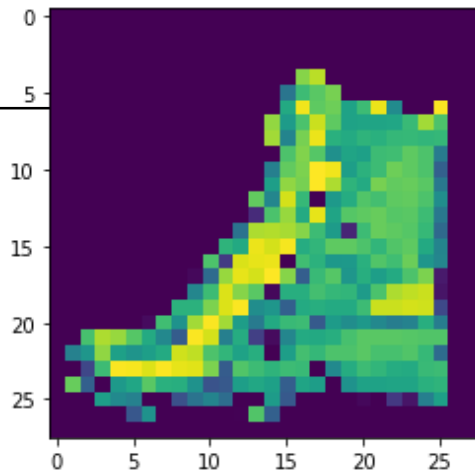
좌우대칭

학습된 2층 신경망 모델로 발목 부츠 이미지를 입력하여 예측해보자(원본 이미지)

다음은 좌우 전환된 발목 부츠 이미지로 학습된 2층 신경망 모델에 적용하여 예측을 수행하는 파이썬 코드이다.

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']  
yhat = np.argmax(model.predict(input_mirror))  
print(class_names[yhat])    # Ankle boot
```

실행 결과, 발목 부츠(Ankle boot)으로
정확하게 예측한 것을 알 수 있음



1 입력으로 주어진 데이터에 따라 한계를 가질 수밖에 없음

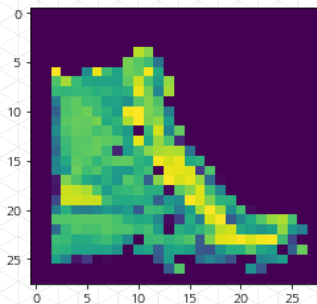
- ☞ 훈련용으로 주어진 데이터에서 모든 신발은 왼쪽을 쳐다보고 있었기 때문에 오른쪽을 쳐다보는 발목 부츠가 들어왔을 때, 이것이 신발류라는 것을 인식하지 못하는 것임

2 패션 MNIST 데이터의 경우 모두 검정으로 처리되어 있기 때문에 또 다른 문제가 발생할 수 있음

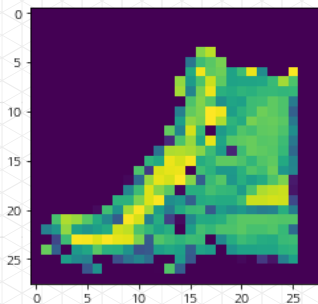
- ☞ 배경이 검정색이 아닌 이미지를 입력하면, 이 이미지가 명백히 신발임에도 불구하고 우리가 만들어낸 모델이 운동화나 부츠로 인식하지 못할 가능성이 높음
- ☞ 이렇게 전체를 대표하지 못하고 특정한 특징이나 경향을 가진 데이터만 지나치게 학습에 많이 사용되는 것을 데이터 편향(Data bias)이라고 함

3 데이터 편향은 학습된 모델이 바르지 못한 판단을 내리게 만들

- 같은 객체라도 방향에 따라 왼쪽은 가방(Bag), 오른쪽은 발목 부츠(Ankle Boots)로 인식함



가방(오른쪽을 쳐다봄)



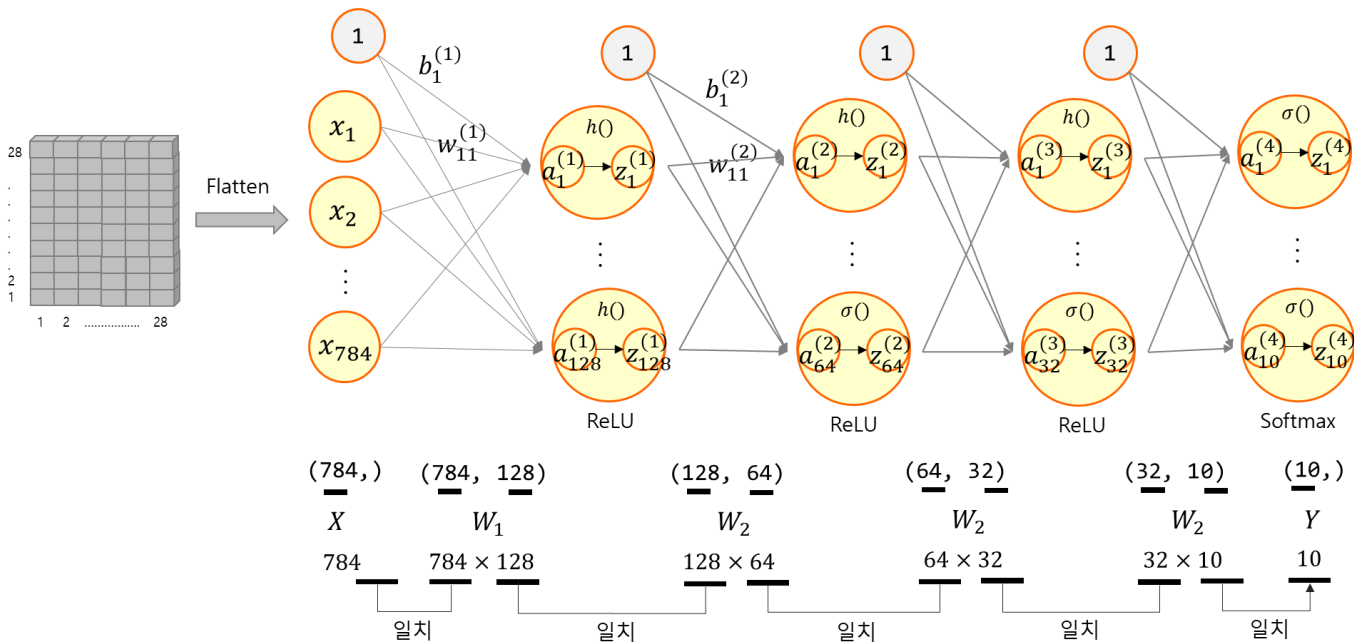
발목 부츠(왼쪽을 쳐다봄)

- 이러한 문제를 해결하기 위해서는 편향되지 않은 데이터를 확보하는 것이 중요함
- 하지만 다양한 데이터를 확보하는 데에 한계가 있을 수 있음

이런 경우에 확보된 데이터를 다양하게 변형하여 데이터 편향을 제거하는 방법이 있는데, 이를 **데이터 증강(Data augmentation)**이라고 함

패션 MNIST 데이터셋을 이용한 4층 완전연결층 인공신경망 구조를 만들어보자.

4층 인공신경망 계층구조는 다음과 같음



인공신경망 각 층의 배열 형상



패션 MNIST 데이터셋을 이용한 4층 완전연결층 인공신경망 구조를 만들어보자.

- ➡ 텐서플로 케라스의 Dense 클래스는 학습을 위한 연결을 밀집된(Dense) 구조 혹은 완전연결층(Fully-connected)으로 함
- ➡ Dense 클래스로 4개의 완전연결층을 가진 네트워크 모델을 생성함
 - 1 Flatten 계층 : 2차 입력 (28, 28)을 1차원 784(28×28)으로 변경
 - 2 Dense 네트워크 : 출력 128, 활성화함수 = ReLU
 - 3 Dense 네트워크 : 출력 64, 활성화함수 = ReLU
 - 4 Dense 네트워크 : 출력 32, 활성화함수 = ReLU
 - 5 Dense 네트워크 : 출력 10, 활성화함수 = SoftMax



패션 MNIST 데이터셋을 이용한 4층 완전연결층 인공신경망 구조를 만들어보자.

4층 신경망 계층구조

```
model2 = keras.Sequential([  
    keras.layers.Flatten(input_shape=(28, 28)),  
    keras.layers.Dense(128, activation='relu'),  
    keras.layers.Dense(64, activation='relu'),  
    keras.layers.Dense(32, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```



다음은 `compile()` 함수로 모델 학습 과정을 설정하는 파이썬 코드이다.

```
model2.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
               metrics=['accuracy'])
```

- 신경망 학습은 기본적으로 추측을 한 뒤에 정답과 비교하여 오차가 얼마인지 확인한 뒤에 이 오차를 줄이는 방법으로 연결의 강도를 조절하는 것임
- 모델을 점점 더 좋은 상태로 만드는 것을 최적화(Optimization)라고 부름
(위 예제에서는 아담(Adam)을 사용)
- 현재 모델의 예측값과 실제 정답의 차이가 오차이고, 이 오차를 측정하는 것이 손실함수
(위 예제에서는 `sparse_categorical_crossentropy` 함수를 사용)
- 위 코드에서는 모델 평가 지표로 정확도(Accuracy)를 사용



다음은 `compile()` 함수로 모델 학습 과정을 설정하는 파이썬 코드이다.

```
model2.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
               metrics=['accuracy'])
```

`keras.Model.compile()` 함수의 중요한 세 개의 파라미터

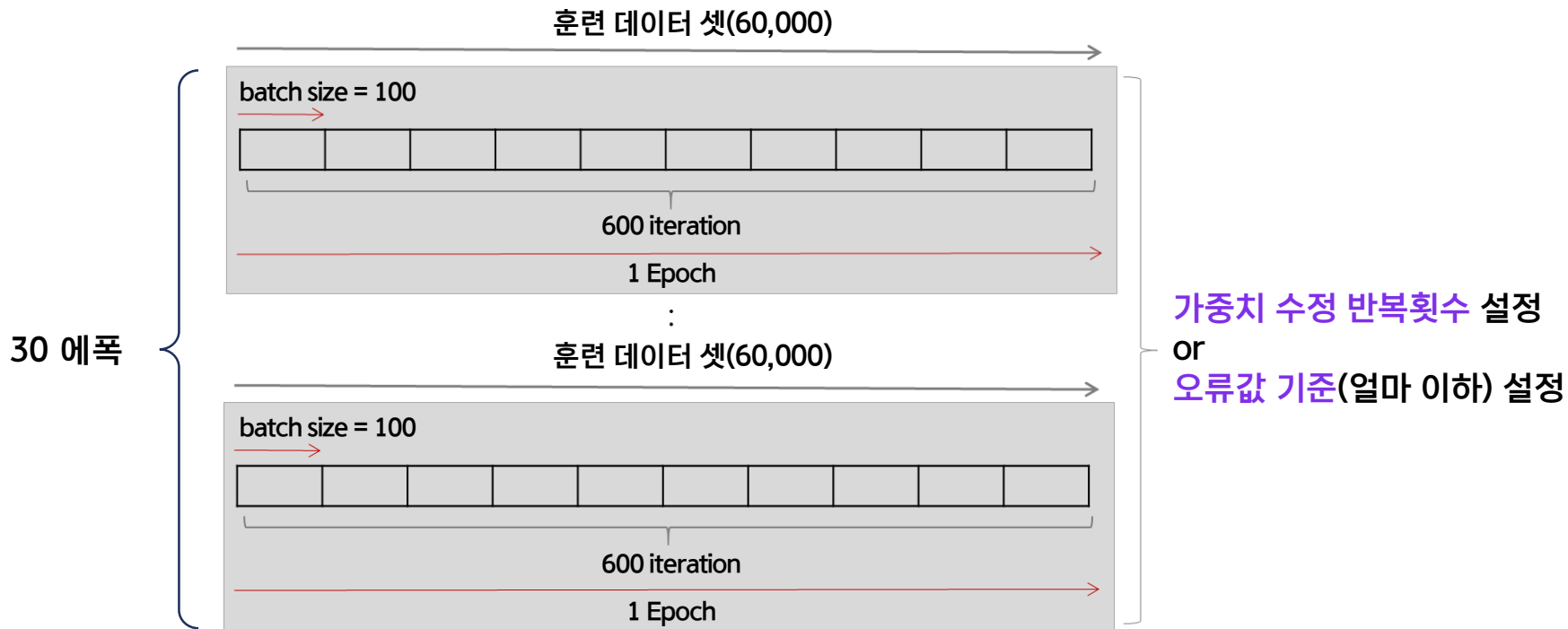
Loss	최적화 과정에서 최소화될 손실함수를 설정하는 것(MSE 등)
Optimizer	훈련 과정을 설정하는 것(Adam, SGD 등)
Metrics	훈련을 모니터링하기 위해 사용(Accuracy 등)

다음은 `fit()` 함수로 모델을 학습시키는 파이썬 코드이다.

```
model2.fit(train_images, train_labels, epochs=30)
```

- ➡ 모델이 완성되면 **훈련 데이터**와 **정답 데이터**를 주고 **학습**을 실시함
- ➡ **학습**을 시작하게 하는 함수는 모델의 `fit()` 메소드인데, **훈련용 입력**과 여기에 **대응**하는 **정답 레이블** 데이터 셋을 **차례로 설정**하면 됨
- ➡ **훈련 데이터** 모음을 가지고 **한 번 훈련**을 실시하는 것을 **에폭**(Epoch)이라고 부름
- ➡ 위 코드에서는 **30개의 에폭**을 지정하였으므로 **훈련**을 **30차례** 실시하는 것임

다음은 `fit()` 함수로 모델을 학습시키는 파이썬 코드이다.





다음은 생성된 모델로 패션 MNIST 데이터를 30에폭 학습한 결과이다.

```
1875/1875 [=====] - 5s 3ms/step - loss: 0.3195 - accuracy: 0.8849
Epoch 16/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.3172 - accuracy: 0.8860
Epoch 17/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.3114 - accuracy: 0.8865
Epoch 18/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.3131 - accuracy: 0.8865
Epoch 19/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.3013 - accuracy: 0.8902
Epoch 20/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2983 - accuracy: 0.8917
Epoch 21/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2968 - accuracy: 0.8910
Epoch 22/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2997 - accuracy: 0.8911
Epoch 23/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2951 - accuracy: 0.8935
Epoch 24/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2949 - accuracy: 0.8945
Epoch 25/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2847 - accuracy: 0.8975
Epoch 26/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2870 - accuracy: 0.8957
Epoch 27/30
1875/1875 [=====] - 6s 3ms/step - loss: 0.2837 - accuracy: 0.8979
Epoch 28/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2874 - accuracy: 0.8952
Epoch 29/30
1875/1875 [=====] - 5s 3ms/step - loss: 0.2781 - accuracy: 0.8985
Epoch 30/30
1875/1875 [=====] - 6s 3ms/step - loss: 0.2805 - accuracy: 0.8988
```

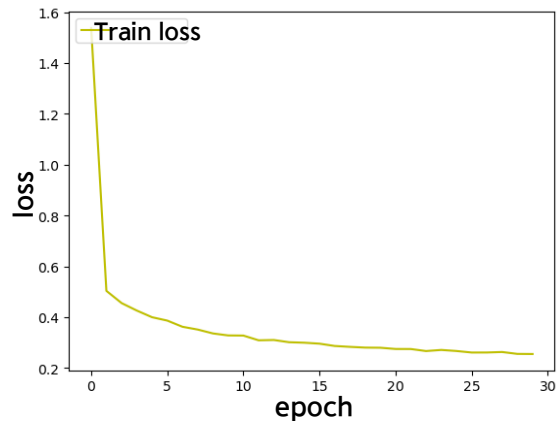


다음은 모델로 패션 MNIST 데이터를 학습한 결과를 시각화한 것이다.

```
import matplotlib.pyplot as plt

fig, loss_ax = plt.subplots()
loss_ax.plot(hist2.history['loss'], 'y', label='train loss')
loss_ax.set_xlabel('epoch')
loss_ax.set_ylabel('loss')
loss_ax.legend(loc='upper left')
plt.show()
```

결과를 살펴보면 모델이
훈련 데이터를 잘 학습하고 있음을 볼 수 있음





다음은 학습된 모델 평가를 위해 `evaluate()` 함수와 테스트 데이터로
모델 평가를 수행하는 파이썬 코드이다.

```
test_loss, test_acc = model2.evaluate(test_images, test_labels, verbose=2)

print('\n테스트 정확도:', test_acc) # 테스트 정확도 : 0.8640000224113464
```

- 실행 결과 정확도는 86.4% 인 것을 알 수 있음
- 즉, 층을 쌓아(2층 → 4층) 정확도(81.37% → 86.4%)가 개선된 것을 알 수 있음

다음은 층을 쌓아(2층에서 4층으로) 얼마나 좋아졌는지 혼동행렬(사이킷런의 `confusion_matrix()`)을 이용하여 각각의 모델이 예측한 레이블과 정답 레이블을 비교해보자.

다음은 두 개의 모델로 테스트 이미지로 예측한 결과를 혼동행렬로 출력하는 파이썬 코드이다.

```
pred1 = model.predict(test_images)    # model 예측
pred2 = model2.predict(test_images)   # model2 예측

y_hat1 = np.argmax(pred1, axis=1)     # model 예측
y_hat2 = np.argmax(pred2, axis=1)     # model2 예측

from sklearn.metrics import confusion_matrix

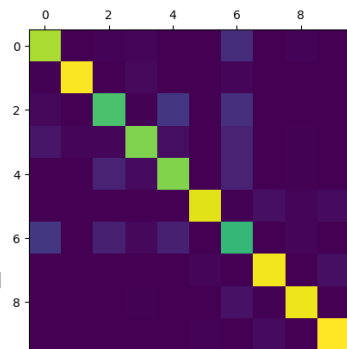
matrix1 = confusion_matrix(test_labels, y_hat1)
matrix2 = confusion_matrix(test_labels, y_hat2)

plt.matshow(matrix1)
plt.matshow(matrix2)
```

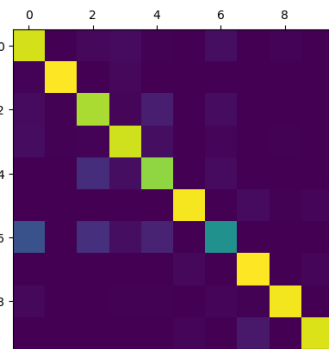
다음은 층을 쌓아(2층에서 4층으로) 얼마나 좋아졌는지 혼동행렬(사이킷런의 confusion_matrix())을 이용하여 각각의 모델이 예측한 레이블과 정답 레이블을 비교해보자.

- 아래 그림과 같이 층을 더 쌓아 학습을 시켰을 때에 눈에 띄는 변화는 대부분 클래스 인식을 제대로 하는 경우가 대폭 늘어난 것임
- 그러나 클래스 6에 속하지 않는데 이미지 6으로 잘못 분류한 경우가 오히려 늘어난 것을 볼 수 있음
- 전반적으로 층을 쌓음으로써 모델의 학습이 더 잘 된 것을 확인할 수 있음

```
[[837  2 10 17  0  0 126  0  8  0]
 [  4 954  1 25  2  0 13  0  1  0]
 [ 20  3 683  6 152  0 133  0  3  0]
 [ 59 13 16 776 35  0 97  0  4  0]
 [  0  0 93 29 777  0 96  0  5  0]
 [  0  1  0  0  0 916  4 38 14 27]
 [155  1 83 19 85  0 641  0 16  0]
 [  0  0  0  0  0 14  2 943  2 39]
 [  3  0  0  5  2  2 47  5 936  0]
 [  0  0  0  0  0  8  1 30  1 960]]
```



2층 인공신경망



4층 인공신경망

```
[[903  1 19 29  4  0 36  0  8  0]
 [  9 965  2 19  3  0  2  0  0  0]
 [ 28  5 843 12 80  0 31  0  1  0]
 [ 31  7 11 898 35  0 14  0  4  0]
 [  1  0 128 34 808  0 29  0  0  0]
 [  0  0  0  0  0 954  0 27  5 14]
 [245  0 130 37 97  0 489  0  2  0]
 [  0  0  0  0  0 20  0 967  0 13]
 [ 19  1  1  7  4  1 14  4 949  0]
 [  0  0  0  1  0 15  1 66  0 917]]
```




다음은 학습된 4층 신경망 모델을 save() 함수로 저장하는 파이썬 코드이다.

```
import os

model2.save(os.getcwd()+'/model/fashion_dnn4_model.h5')
print("Saved model to disk !!!")
```

모델이 현재작업경로 아래의 “/model/fashion_dnn4_model.h5” 파일로 저장된 것을 알 수 있음

다음은 저장된 4층 신경망 모델을 load_model() 함수로 읽어오는 파이썬 코드이다.

```
import os
from tensorflow.keras.models import load_model

model2 = load_model(os.getcwd()+'/model/fashion_dnn4_model.h5')
model2.summary()
```

오른쪽 그림은 **인공신경망 모델의
네트워크 구조**를 나타낸 것임

Model: "sequential_3"

Layer (type)	Output Shape	Param #
flatten_3 (Flatten)	(None, 784)	0
dense_8 (Dense)	(None, 128)	100480
dense_9 (Dense)	(None, 64)	8256
dense_10 (Dense)	(None, 32)	2080
dense_11 (Dense)	(None, 10)	330

=====
Total params: 111,146
Trainable params: 111,146
Non-trainable params: 0
=====

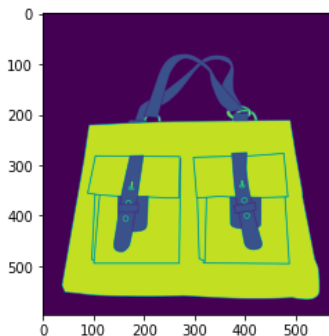
학습된 4층 인공신경망 모델로 가방 이미지를 입력하여 예측해 보자.

다음은 OpenCV를 이용해 만화 같은 가방 이미지를 회색조 단일 채널로 변환하여 화면에 출력하는 파이썬 코드이다.

```
import cv2
import os

img2 = cv2.imread(os.getcwd()+'/images/bag_cartoon.png', cv2.IMREAD_GRAYSCALE)
plt.imshow(img2)
print(img2.shape)          # 형상 (596, 576)
```

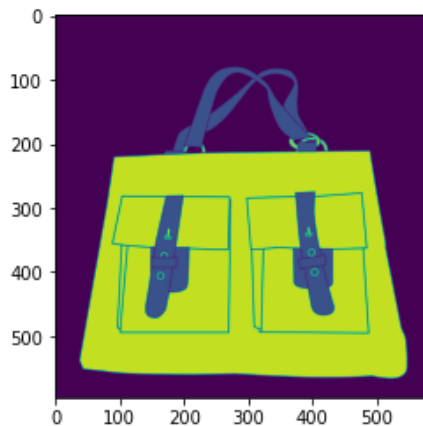
그림과 같이 (596, 576) 형상의
만화 같은 가방 이미지가 출력된 것을 볼 수 있음



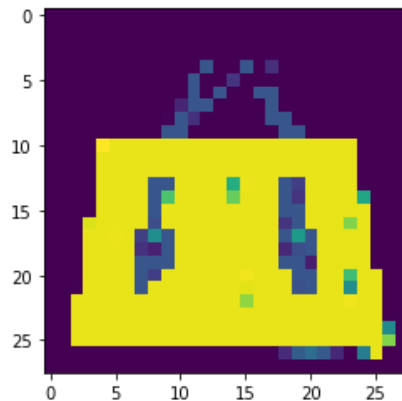
다음은 만화 같은 가방 이미지의 형상을 변경((596,576) → (28,28))하는 코드이다.

```
img2 = cv2.resize(img2, (28, 28))  
plt.imshow(img2)
```

아래 그림과 같이 (28, 28) 형상의 만화 같은 가방 이미지로 변경된 것을 알 수 있음



(596, 576)



(28, 28)



텐서플로를 이용한 4층 완전연결층 프로그래밍

다음은 만화 같은 가방 이미지의 형상을 3차원으로 늘려서 모델에 입력하여 예측을 수행한다.

```
# 클래스 이름을 리스트로 만들기
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

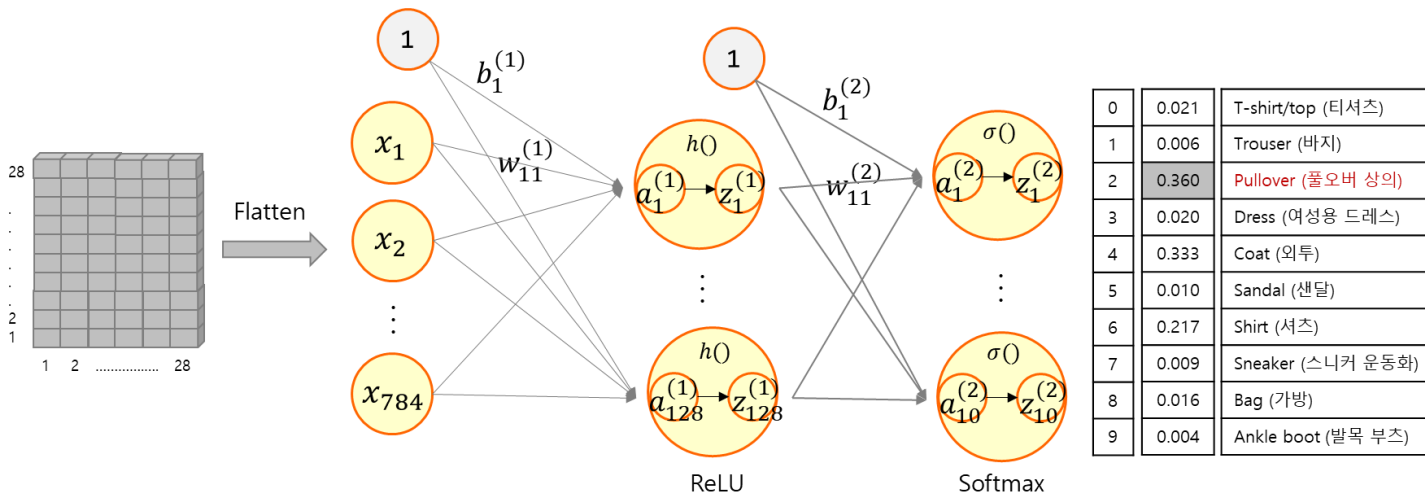
input_data2 = img2[np.newaxis, :, :]
yhat2 = np.argmax(model2.predict(input_data2)) # 저장된 모델 : 만화 같은 가방 이미지 인식

print(class_names[yhat2]) # Bag
```

실행 결과, 정확히 잘 예측한 것을 볼 수 있음

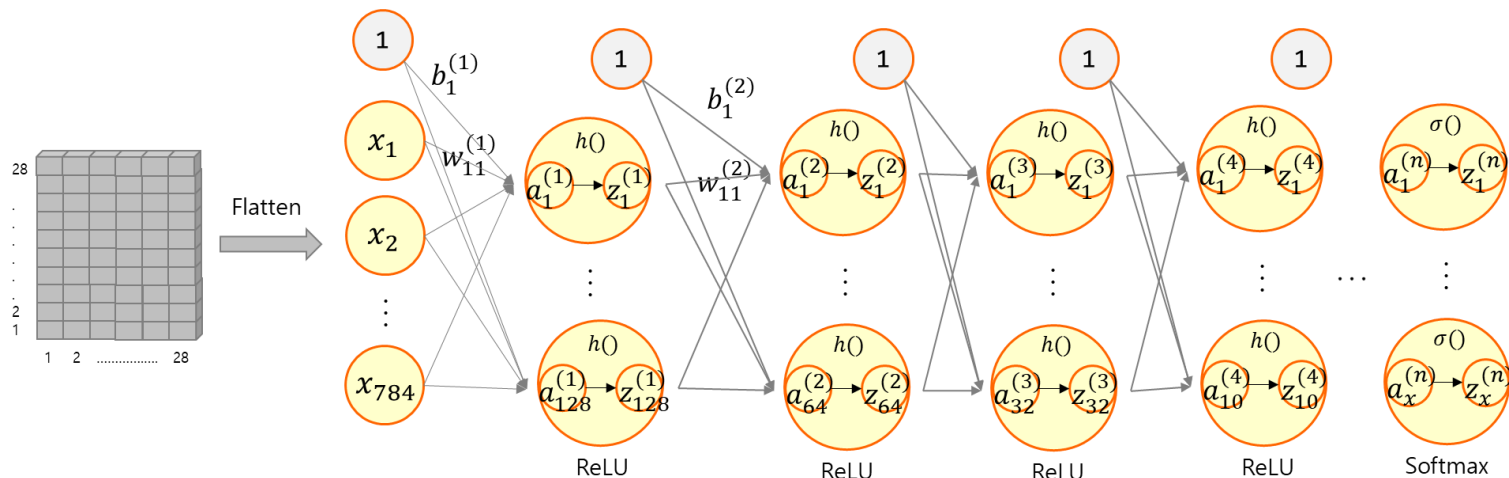
■ 대표적인 문제는 모델이 복잡해서 계산 시간이 많이 소요되는 것이 대표적인 문제

- ➡ 더 중요한 문제는 단순한 구조로 층만 깊이 쌓을 경우, 출력 부분의 잘못에 근거한 학습의 내용이 입력에 가까운 네트워크 층까지 전파가 잘 되지 않는다는 것임
- ➡ 즉, 2층 인공신경망에서 N층으로 은닉층을 쌓기만 하면 학습이 잘 되지 않음



2층 인공신경망 구조

- 따라서, 층이 깊어질수록 입력에 따른 동작을 내도록 조정하기 어려워진다는 문제도 발견됨
- 이것은 층을 높이 쌓을수록 학습이 제대로 되지 않는다는 것임
- 이러한 이유로 심층 구조는 학습이 이루어진다 해도 과적합이 되는 특성을 가짐



N층 인공신경망 구조

■ 합성곱 신경망(Convolution Neural Network, CNN) 모델(AlexNet, 2012)

- 심층 신경망을 효과적으로 학습시킬 수 있는 돌파구를 보여줌
- 연구자 : 제프리 힌튼(Geoffrey Hinton), 요수아 벤지오(Yoshua Bengio), 얀 르쿤(Yann LeCun) 등

